

Autodesk® Scaleform®

Scaleform 4.3

Scaleform 4.3

Michael Antonov, Bart Muzzin

1.05

2013 4 8

Copyright Notice

Autodesk® Scaleform® 4.3

© 2013 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk 123D, Autodesk Homestyler, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, Beast (design/logo), BIM 360, Built with ObjectARX (design/logo), Burn, Buzzsaw, CADmep, CAiCE, CAMduct, CFdesign, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, Design Server, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWGX, DXF, Ecotect, ESTmep, Evolver, Exposure, Extending the Design Team, FABmep, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, ForceEffect, Freewheel, GDX Driver, Glue, Green Building Studio, Heads-up Design, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, iMOUT, Incinerator, Inferno, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, Map It, Build It, Use It, MatchMover, Maya, Mechanical Desktop, MIMI, Moldflow, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPI (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, PolarSnap, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, Revit LT, RiverCAD, Robot, Scaleform, Scaleform GFx, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Socialcam, Sparks, SteeringWheels, Stitcher, Stone, StormNET, TinkerBox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, U-Vis, ViewCube, Visual, Visual LISP, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

How to Contact Autodesk Scaleform:

Document	Scaleform 4.3 Renderer Threading Guide
Address	Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA
Website	www.scaleform.com
Email	info@scaleform.com
Direct	(301) 446-3200
Fax	(301) 446-3199

1		1
2		1
2.1		1
2.2		2
2.3		3
2.4		5
2.5		6
3		7
3.1		7
3.2		8
4		9
5		11
5.1	TextureCache	11
6		12
6.1		12
6.2		12
6.3		14
7	GfxShaderMaker	15
7.1	一般说明	15
7.2	支持多个着色器 API 的 HAL	15
7.2.1	D3D9	15
7.2.2	D3D1x	15

1



2

API Scaleform 3.x Scaleform 4.0

1. Renderer2D
2. GFXLoader
- 3.
4. MovieDisplayHandle
5. GRenderer Scaleform 3.X
6. API Render::HAL

2.1

Scaleform Render::Renderer2D Render::HAL Renderer2D
Display Render::HAL Renderer2D
" " Render::D3D9::HAL Render::PS3::HAL

```
D3DPRESENT_PARAMETERS PresentParams;  
IDirect3DDevice9* pDevice = ...;  
ThreadId renderThreadId = Scaleform::GetCurrentThreadId();  
Render::ThreadCommandQueue *commandQueue = ...;  
Ptr<Render::D3D9::HAL> pHal;  
Ptr<Render::Renderer2D> pRenderer;
```

```
pHal = *SF_NEW Render::D3D9::HAL(commandQueue);
if (!pHal-> InitHAL(Render::D3D9::HALInitParams(pDevice, PresentParams, 0,
renderThreadId)))
{
    return false;
}
pRenderer = *SF_NEW Render::Renderer2D(pHal);
```

HALInitParams class
renderThreadId
ThreadCommandQueue
pDevice PresentParams

Renderer2D HAL

2.2

Scaleform 4.0
Scaleform 3.x
Render::HAL
Renderer2D
GfxLoader
ShutdownHAL
InitHAL

Render::HAL HAL

```
MeshCacheParams mcp;
mcp.MemReserve = 1024 * 1024 * 3;
mcp.MemLimit = 1024 * 1024 * 12;
mcp.MemGranularity = 1024 * 1024 * 3;
mcp.LRUTailSize = 1024 * 1024 * 4;
mcp.StagingBufferSize = 1024 * 1024 * 2;

pRenderer->GetMeshCacheConfig()->SetParams(mcp);
```

MemReserve
MemLimit
MeshCache
MemGranularity
LRU
LRUTailSize
MeshCache
StagingBufferSize
64K
PC

```

GlyphCacheParams gcp;
gcp.TextureWidth = 1024;
gcp.TextureHeight = 1024;
gcp.NumTextures = 1;
gcp.MaxSlotHeight = 48;

pRenderer->GetGlyphCacheConfig()->SetParams(gcp);

```

Scaleform 3.3	TextureWidth	TextureHeight	NumTextures
	MaxSlotHeight		

InitHAL

InitHAL

SetParams

2.3

Scaleform 4.0

Gfx::ImageCreator
Scaleform

Render::TextureManager
ImageCreator Gfx::Loader

```

Gfx::Loader loader;
...

SF::Ptr<Gfx::ImageCreator> pimageCreator =
    *new Gfx::ImageCreator(pRenderThread->GetTextureManager());
loader.SetImageCreator(pimageCreator);

```

ImageCreator

TextureManager

TextureManager

Render::HAL::GetTextureManager()
GetTextureManager

Render::HAL

TextureManager

CreateTexture

-

D3DCREATE_MULTITHREADED

D3D9

-

ThreadCommandQueue

Render::HAL

D3D

ThreadCommandQueue

Platform::RenderHALThread

2.4

The diagram illustrates the relationship between various components in the Gfx::MovieDisplayHandle namespace. It shows a hierarchy where Gfx::MovieDisplayHandle is the parent of Scaleform 4.0 APIs, Gfx::Movie, and Gfx::MovieDef::CreateInstance. Gfx::Movie is further associated with Gfx::MovieDef::CreateInstance. The diagram also shows a relationship between Gfx::Movie and Gfx::MovieDef::CreateInstance, and a relationship between Gfx::Movie and Gfx::MovieDef::CreateInstance.

```

graph TD
    GfxMovieDisplayHandle[Gfx::MovieDisplayHandle] --- Scaleform40APIs[Scaleform 4.0 APIs]
    GfxMovieDisplayHandle --- GfxMovie[Gfx::Movie]
    GfxMovieDisplayHandle --- GfxMovieDefCreateInstance[Gfx::MovieDef::CreateInstance]
    GfxMovie --- GfxMovieDefCreateInstance
    GfxMovie --- GfxMovieDefCreateInstance
    GfxMovie --- GfxMovieDefCreateInstance
  
```

```
//-----  
// 1.  
  
Ptr<GFx::Movie>          pMovie = ...;  
GFx::MovieDisplayHandle  hMovieDisplay;  
  
//  
//  
pMovie->SetViewport(width, height, 0,0, width, height);  
hMovieDisplay = pMovie->GetDisplayHandle();  
  
//                               Scaleform Player  
//  
pRenderThread->AddDisplayHandle(hMovieDisplay);  
  
//-----  
// 2.  
  
//  
//      ExternalInterface          Advance  
float deltaT = ...;  
pMovie->HandleEvent(...);  
pMovie->Advance(deltaT);
```

```

//
//
pRenderThread->WaitForOutstandingDrawFrame();
pRenderThread->DrawFrame();

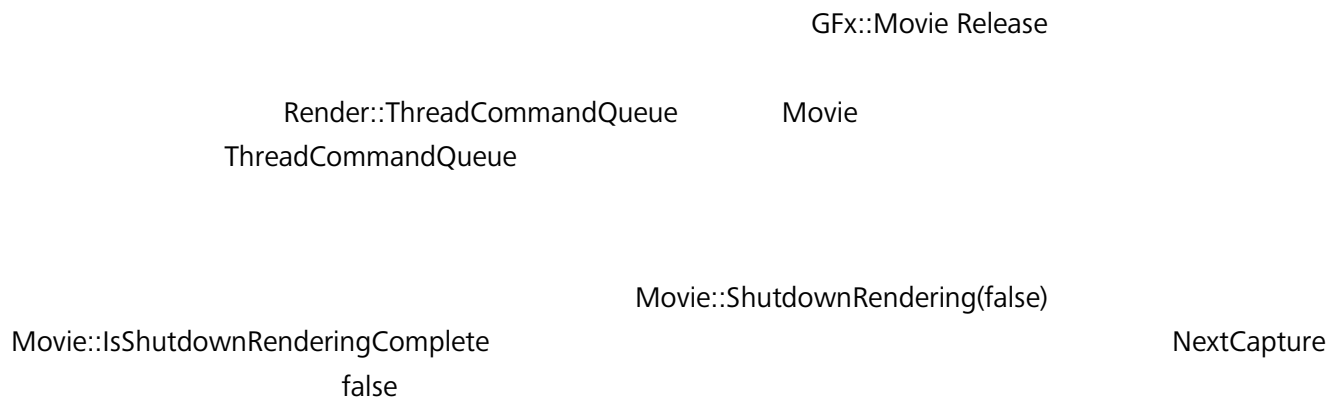
//-----
// 3.         DrawFrame
Ptr<Render::Renderer2D> pRenderer = ...;

pRenderer->BeginFrame();
bool hasData = hMovieDisplay.NextCapture(pRenderer->GetContextNotify());
if (hasData)
    pRenderer->Display(hMovieDisplay);
pRenderer->EndFrame();

```

- WaitForOutstandingFrame – CPU
- Advance
- MovieDisplay.NextCapture – “ Advance false

2.5



3

Scaleform

—

GFX::Movie::Advance

ActionScript

Advance

Advance

Scaleform

Advance

Render::HAL

Scaleform

•

•

Scaleform Player

Scaleform

3.1

Advance

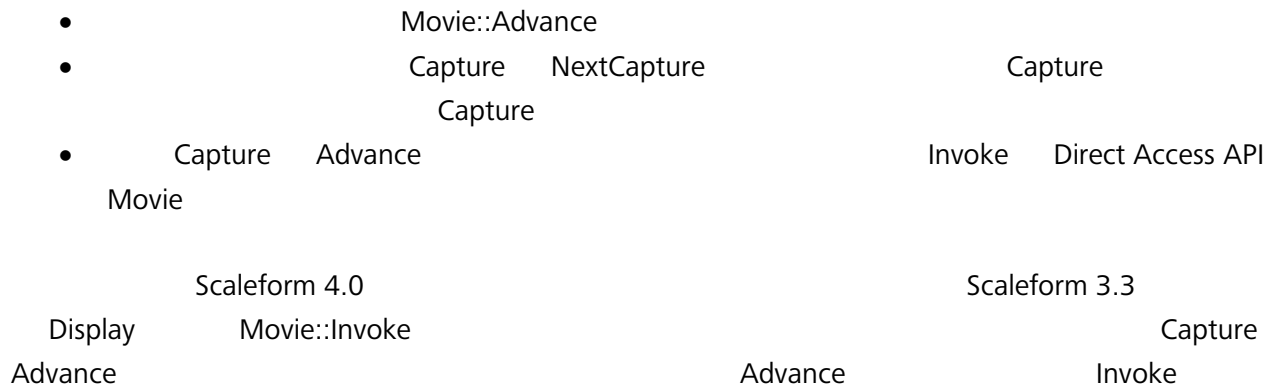
/

Scaleform

GFX::Movie::Advance

GFX::Movie::Capture

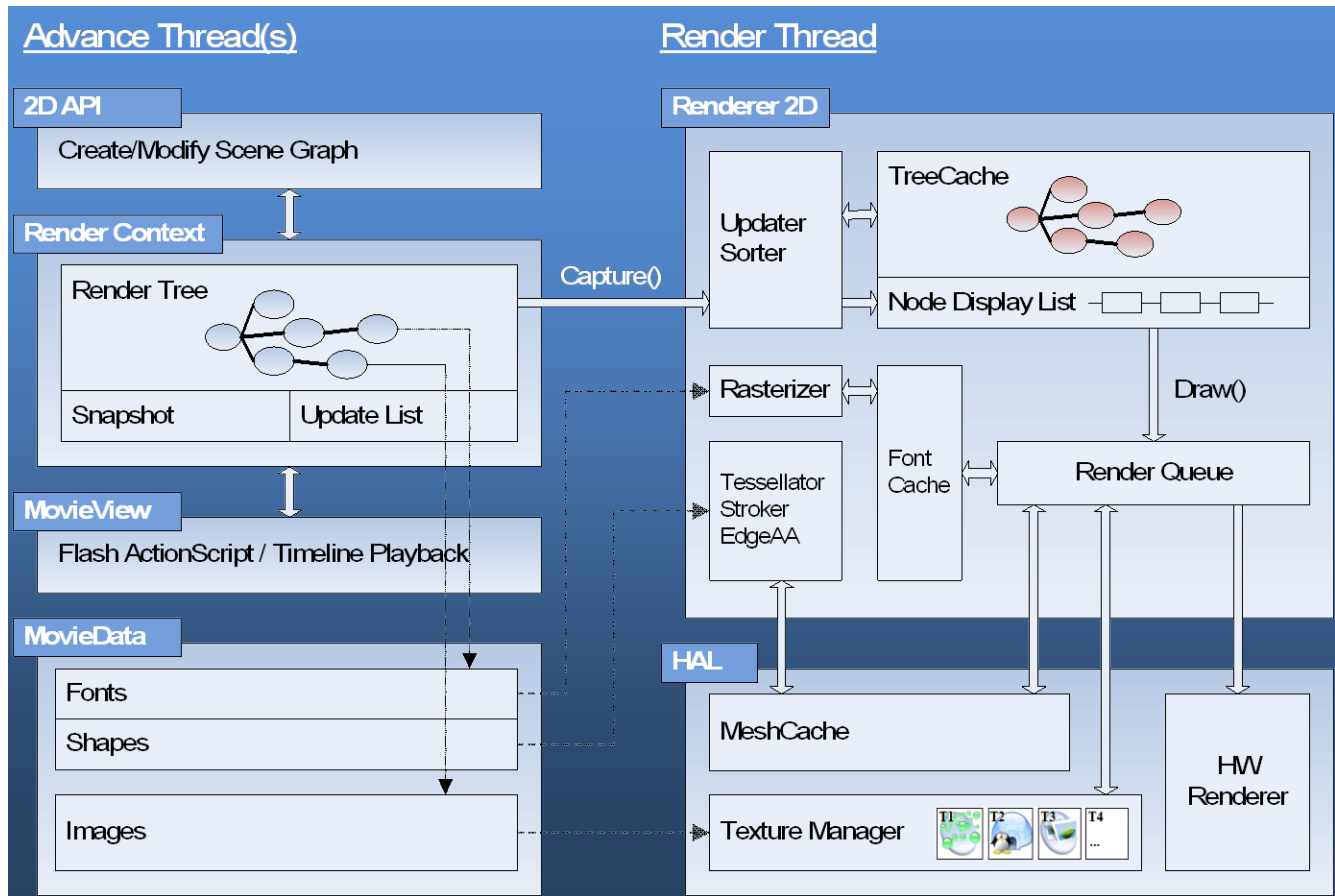
NextCapture



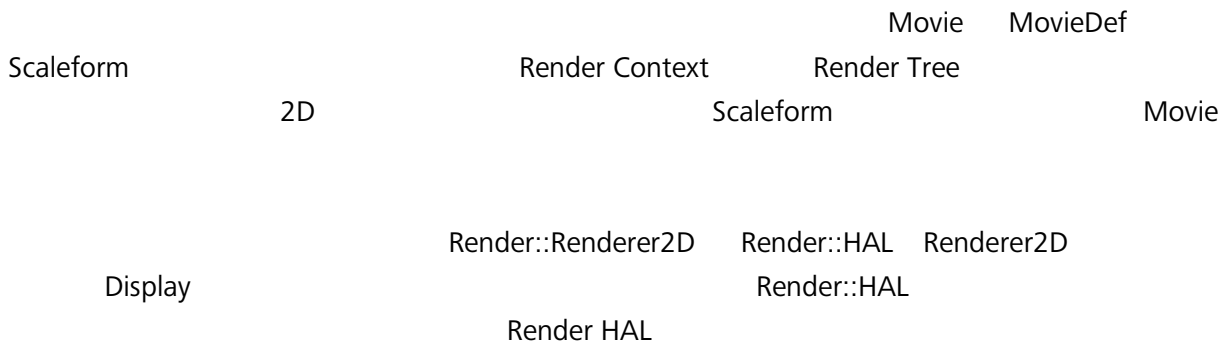
3.2

Render::Renderer2D::Display 进行设备调用来在渲染设备上渲染电影的帧。出于性能原因，不保留混合模式和纹理存储设置等各种设备状态，而且调用 Render::Renderer2D::Display 之后设备的状态会发生变化。

Scaleform



1



-
- Font Cache Font Cache
- Mesh Cache
- Render Queue Render Queue /
CPU
- HAL API

ActionScript AI
/

5

5.1 TextureCache

TextureCache 系统处理卸载纹理资源 的问题。该系统尽管可用于各种平台，但它更适用于内存受限的系统，例如手机平台，目的是为了减少总体内存占用量。不过，只有在 `HAL::InitHAL` 内隐式创建 `TextureManager` 对象时，默认情况下才在 iOS、Android 和 PS Vita 平台上初始化 TextureCache 系统。TextureCache 仅适用于图像数据小于纹理数据的图像，即压缩的图像必须解码成 GPU 使用的未压缩的 RGBA 颜色数据。其中包括 PNG、JPEG 和 Flash 内部图像格式。TextureCache 不适用于经过压缩并可由 GPU 直接使用的图像，也不适用于未经压缩的图像 – 一旦从图像中复制其纹理数据，这些图像的数据就会被自动卸载。

假如将 `TextureManager` 分配并传递到 `HALInitParams` 之中，`TextureManager` 就会将 `TextureCache` 作为其构造函数的最后一个参数。只提供 `TextureCache` 的一个实现，名为 `TextureCacheGeneric`，不过，用户可以创建一个派生的 `TextureCache` 实现。如果是这种情况，就必须分配 `TextureManager` 并传递一个派生的 `TextureCache` 实例，然后必须通过 `HALInitParams` 将 `TextureManager` 传递到 `InitHAL` 之中。

`TextureCache` 的一般实现采用一种从内存中驱逐出纹理的简单 LRU 策略。而且，只有给适用的纹理分配一个阈值之后才会驱逐纹理。一旦达到该阈值（默认情况下为 8MB），就会驱逐 LRU 纹理，直到再次低于限值。如果无法达到限值，因为单个帧中使用纹理内存的不止一个阈值，就会发出警告。运行时，可以用 `TextureCache` 的 `SetLimitSize` 函数调节该限值。系统从不会驱逐正常渲染所必需的纹理。

6

用户偶尔希望通过与在 Scaleform 标准版本中可用的方法不同的方法来渲染某些形状。这可能是达到改变的顶点转换或特殊片段着色器填充效果的一种方法。

6.1

一般情况下，只有电影内小的形状子集需要特殊渲染效果。在 ActionScript 中，这些对象是通过使用 AS2 扩展属性 “rendererString” 和 “rendererFloat” 或者 AS3 扩展方法 “setRendererString” 和 “setRendererFloat” 进行标识的。请参阅 “AS2/AS3 扩展” 参考，了解有关其使用方法的详细信息。渲染一个电影剪辑之前，在该电影剪辑上设置一个字符串和/或浮点（Float），会导致通过 HAL::PushUserData 函数将 UserDataState::Data 的一个实例推送到 Render::HAL 的 UserDataStack 成员上。整个堆栈可随时访问，因而允许使用嵌套的数据，不过，每个电影剪辑只能包含一个 “字符串” 和/或 “浮动”。当该电影剪辑完成渲染时，就会将 UserDataState::Data 从 HAL::PopUserData 函数弹出堆栈。

6.2

目前，要执行自定义渲染，同时将电影剪辑上设置的用户数据考虑在内，就必须修改 HAL。尽管为执行自定义渲染而修改 HAL 的方法不少，但本示例还是将重点放在对内部着色器系统的简单扩展上，方法是将一个 2D 位置偏移量添加到所渲染的具有相应用户数据的所有形状中。

对于本示例，创建一个 AS3 SWF，它具有一个实例名为 “m” 的电影剪辑以及下面的 AS3 代码：

```
import scaleform.gfx.*;
DisplayObjectEx.setRendererString(m, "Abc");
DisplayObjectEx.setRendererFloat(m, 17.1717);
```

利用 4.1 中的新的着色器脚本编写系统，我们可以容易地添加新的着色器特性。在 Src/Render/ShaderData.xml 中，添加下面突出显示的一行：

```
<ShaderFeature id="Position">
  <ShaderFeatureFlavor id="Position2d" hide="true"/>
  <ShaderFeatureFlavor id="Position3d"/>
  <ShaderFeatureFlavor id="Position2dOffset"/>
</ShaderFeature>
```

该行将使着色器系统使用 “Position2dOffset” 片段，以便实现处理顶点着色器中的位置数据。然后再在文件中添加 “Position2dOffset” 片段：


```
<ShaderSource id="Position2dOffset" pipeline="Vertex">
    Position2d(
        attribute float4 pos      : POSITION,
        varying   float4 vpos     : POSITION,
        uniform   float4 mvp[2],
        uniform   float  poffset)
    {
        vpos = float4(0,0,0,1);
        vpos.x = dot(pos, mvp[0]) + poffset;
        vpos.y = dot(pos, mvp[1]) + poffset;
    }
</ShaderSource>
```

此片段与 “Position2d” 片段相同 – 除此片段将统一值 ‘poffset’ 添加到顶点的转换后 x 和 y 之外。

现在，在 D3D9_Shader.cpp 中，将下列代码块添加到 ShaderInterface::SetStaticShader 的最上面：

```
//
    if (pHal->UserDataStack.GetSize() > 0 )
    {
        const UserDataState::Data* data = pHal->UserDataStack.Back();
        if (data->Flags &
(UserDataState::Data::Data_Float|UserDataState::Data::Data_String) &&
            data->RendererString.CompareNoCase("abc") == 0)
        {
//
            vshader = (VertexShaderDesc::ShaderType)((int)vshader +
VertexShaderDesc::VS_base_Position2dOffset);
            shader  = (FragShaderDesc::ShaderType) ((int)shader +
FragShaderDesc::FS_base_Position2dOffset);
        }
    }
```

此代码块检查 HAL 的 UserDataStack 是否具有我们要在最上面拦截的数据（有一个 “abc” 字符串，并且还有一个浮点值）。如果匹配，我们就修改要在 Position2dOffset 路径中添加的传入的着色器类型。请注意，在通过自定义构建步骤（生成新的 D3D9_ShaderDescs.h）运行 ShaderData.xml 之前，这些符号不可用。

除更改所用的着色器定义之外，我们还需要在实际渲染之前更新 ‘poffset’ 统一值。这可以通过在 ShaderInterface::Finish 最上面插入下列代码块来完成：

```
//
    if (pHal->UserDataStack.GetSize() > 0 )
    {
        const UserDataState::Data* data = pHal->UserDataStack.Back();
```

```

        if (data->Flags &
(UserDataState::Data::Data_Float|UserDataState::Data::Data_String) &&
        data->RendererString.CompareNoCase("abc") == 0)
        {
//    poffset
            for ( unsigned m = 0; m < Alg::Max<unsigned>(1, meshCount); ++m)
            {
                float poffset = data->RendererFloat / 256.0f;
                SetUniform(CurShaders, Uniform::SU_poffset, &poffset, 1, 0, m);
            }
        }
    }
}

```

请注意，此示例为每个 ‘meshCount’ 设置一次统一值。对于批处理或实例化绘制（上面进行的着色器修改也支持这些绘制），meshCount 将会大于 1。运行 HAL 的修改后的版本，电影剪辑 ‘m’ 现在应稍微向上和向右移动了一点。

6.3

HAL 也可以修改，这样渲染就完全在 Scaleform 着色器系统之外进行。本文不介绍进行这些修改的确切方法，而且修改方法也会因预期结果的不同而不同。

在外部进行渲染时，可以禁用批处理和实例化网格生成。这是因为在 Scaleform 之外创作的着色器并不像 Scaleform 内部的着色器一样普遍支持批处理和/或实例化。要禁用批处理绘制，只需使用 AS2 “disableBatching” 扩展成员或者使用 AS3 “disableBatching” 扩展函数。请参阅 AS2/AS3 扩展参考文档，了解这些扩展的确切语法。

7 GFxShaderMaker

7.1 一般说明

Scaleform 4.1 中引入 GFxShaderMaker，它是一个构造和编译 Scaleform 使用的着色器的实用工具。重新构造渲染器项目（如 Render_D3D9）时，它是作为一个自定义构造步骤在 ShaderData.xml 上运行的。这会产生多个文件，渲染器使用这些文件编译和/或链接渲染器。这些文件会因平台不同而不同。例如，在 D3D9 上，该实用工具创建：

```
Src/Render/D3D9_ShaderDescs.h  
Src/Render/D3D9_ShaderDescs.cpp  
Src/Render/D3D9_ShaderBinary.cpp
```

7.2 支持多个着色器 API 的 HAL

7.2.1 D3D9

D3D9 HAL 同时支持 ShaderModel 2.0 和 ShaderModel 3.0。默认情况下，HAL 中包含了对所有这些特征级的支持。您可以明确地删除对其中任何特征级的支持。这将会为创建着色器描述符和二进制着色器节约出额外大小。而且可以通过使用 GFxShaderMaker ‘-shadermodel’ 的一个选项并提供一个逗号分隔式着色器模型列表来完成。例如：

```
GFxShaderMaker.exe -platform D3D1x -shadermodel SM30
```

这就会删除对 ShaderModel 2.0 的支持。如果您试图通过一个仅支持 ShaderModel 2.0 的设备调用 InitHAL，该工具会发生故障。

7.2.2 D3D1x

D3D1x HAL 支持三个特征级，它们与用来创建设备的不同等级着色器支持相对应。这些特征级包括（给 D3D11 和 D3D10.1 添加必需的前缀）：

```
FEATURE_LEVEL_10_0  
FEATURE_LEVEL_9_3  
FEATURE_LEVEL_9_1
```

默认情况下，HAL 中包含了对所有这些特征级的支持。您可以明确地删除对其中任何特征级的支持。这将会为创建着色器描述符和二进制着色器节约出额外大小。可以通过使用 GfxShaderMaker '-featurelevel' 的一个选项并提供一个逗号分隔式必需特征级列表来完成。例如：

```
GfxShaderMaker.exe -platform D3D1x -featurelevel  
FEATURE_LEVEL_10_0,FEATURE_LEVEL_9_1
```

这就会删除对 FEATURE_LEVEL9_3 的支持。如果您试图通过一个使用 FEATURE_LEVEL_9_3 的设备调用 InitHAL，它就会使用支持的下一个最低特征级（在此情况下为 FEATURE_LEVEL_9_1）。